

PANDUAN BEDAH KODE

# Wahana Rumah Boneka

Buku pegangan tim Kelompok 1 — dari konsep wahana sampai baris kode



Buku ini memetakan setiap hal yang tampil di presentasi (kereta, rel, interaksi, trigger, UI World Space, show tiap section, optimasi WebGL) ke lokasi aslinya di dalam project Unity: file mana, baris berapa, dan cara menjelaskannya dengan tenang saat sesi tanya jawab.

---

Mata kuliah **Virtual Reality** · Dosen **Calvin Mona Sandehang, M.Sc.** · UAS 2026  
**Kelompok 1:** Antonius Harry (Ketua) · Dimas Kurniawan · Deva Agriani · Dharma Satriadi · Izhar Rahman Dwiputra · Halimah Sukmawati

# Isi Buku

---

- 1 Cara Pakai Buku Ini dan Istilah Dasar
- 2 Peran dan Tanggung Jawab Tim
- 3 Peta Proyek
- 4 Urutan Belajar yang Disarankan
- 5 Konsep dan Letaknya di Kode
- 6 Peta Slide ke Kode
- 7 Antisipasi Pertanyaan Dosen
- 8 Checklist Kesiapan per Anggota

Semua nomor baris pada buku ini merujuk kondisi repo per 8 Juli 2026. Kalau kode berubah setelah tanggal itu, cari nama method-nya — strukturnya tetap sama.

# Cara Pakai Buku Ini dan Istilah Dasar

## Cara membaca

- 1 Baca Bab 2** dan temukan kartu perananmu — di situ tertulis bagian presentasi dan area kode yang kamu pegang.
- 2 Pelajari Bab 5** khusus konsep yang ada badge namamu. Buka filenya di Unity/editor teks, cocokkan dengan cuplikan di buku, dan pahami "kenapa"-nya, bukan menghafal.
- 3 Latihan dengan Bab 7 dan Bab 8:** minta teman membacakan pertanyaan dosen, jawab dengan kata sendiri sambil menunjuk file dan baris.

## Ringkasan proyek dalam satu paragraf

"Wahana Rumah Boneka" adalah experience first-person di browser (WebGL): pemain datang ke taman hiburan malam, masuk lobby "Grand Teater Boneka", mengambil tiket, menaiki kereta kencana, lalu kereta berjalan mengikuti rel tertutup melewati lima section boneka bertema (hutan sihir, kotak musik, kamar anak terbengkalai, akuarium raksasa, langit kamar). Benang merahnya: pelan-pelan pemain sadar bahwa dirinya mainan kecil di dunia mainan seorang anak — dan di akhir ride, sang anak tertidur. Semua dibangun dalam SATU scene Unity: `Assets/Scenes/UAS_Main.unity`.

| Fakta kunci           | Nilai                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unity dan target      | Unity 6000.4.6f1, URP, build WebGL ke itch.io (kompresi Brotli)                                                                                      |
| Waypoint rel utama    | 755 titik (WP_0..WP_754), dibangkitkan dari 80 titik desain rute utama, jarak antar titik 0,5 unit                                                   |
| Cabang rel            | Jalur Beruang di S1: 6 titik desain menjadi 42 waypoint; kereta berhenti di WP 73 menunggu pilihan (cabang fisik di WP 77), bergabung lagi di WP 135 |
| Kecepatan kereta      | normal 2.0 · zona lambat 1.1 · maksimum (tahan W) 3.5                                                                                                |
| Interaksi dan trigger | ObjekInteraksi 11 mode (0-10) · ZonaTrigger 6 mode (0-5)                                                                                             |
| Stempel               | 6 slot: 5 section + 1 bintang emas (dua rute S1 sudah pernah dilalui)                                                                                |
| Skala dunia           | area track sekitar 124 x 106 unit (syarat minimal 40 x 10)                                                                                           |
| Jumlah kode           | 52 script runtime + 7 script pola dosen + 66 menu generator editor (Tools/Wahana)                                                                    |

## Istilah dasar Unity (glossary)

| Istilah                    | Arti sederhana                                                                                                                                         |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| GameObject                 | Benda apa pun di scene (kereta, lampu, zona kosong). Wadah untuk komponen.                                                                             |
| Component                  | Kemampuan yang ditempel ke GameObject: Renderer (tampil), Collider (bisa ditabrak), script kita sendiri.                                               |
| Transform                  | Posisi, rotasi, dan skala sebuah GameObject. <code>transform.position</code> = tempatnya di dunia.                                                     |
| Prefab                     | Cetakan GameObject yang bisa dipakai berulang (contoh: boneka, pohon, lampu taman).                                                                    |
| MonoBehaviour              | Kelas dasar semua script kita. Punya method siklus hidup: <code>Awake</code> (bangun), <code>Start</code> , <code>Update</code> (tiap frame).          |
| [SerializeField]           | Membuat field private tetap terlihat di Inspector supaya bisa diatur tanpa mengubah kode.                                                              |
| Collider dan isTrigger     | Bentuk tabrakan. Kalau isTrigger dicentang, benda bisa menembus tapi Unity memberi tahu lewat <code>OnTriggerEnter/Exit</code> .                       |
| Rigidbody (kinematik)      | Penanda benda "fisik". Kinematik = digerakkan script, bukan gravitasi. Trigger hanya terbaca kalau salah satu pihak punya Rigidbody.                   |
| Raycast                    | Sinar tak terlihat dari kamera ke depan; dipakai mendeteksi "objek apa yang sedang ditatap pemain".                                                    |
| CharacterController        | Komponen bawaan Unity untuk gerak jalan first-person yang tidak menembus dinding.                                                                      |
| Coroutine                  | Fungsi yang bisa "menunggu" ( <code>yield return new WaitForSeconds(2)</code> ) tanpa membekukan game — dipakai untuk urutan show.                     |
| Animator vs animasi script | Animator = memutar clip animasi (boneka, pintu). Animasi script = menggerakkan Transform langsung tiap frame (goyangan, orbit, kelip).                 |
| Canvas World Space         | UI yang hidup DI DALAM dunia 3D (papan status di kereta). Lawannya Screen Space Overlay: menempel di layar (dipakai hanya untuk fade/efek).            |
| Material, Emission, URP    | Material = kulit benda. Emission = bagian yang memancarkan cahaya sendiri (glow). URP = pipeline render Unity yang kami pakai.                         |
| Skybox                     | Gambar langit yang membungkus seluruh scene; punya kami dibuat prosedural (3.200 bintang).                                                             |
| Waypoint                   | Titik-titik jalan. Kereta bergerak dari titik ke titik memakai <code>Vector3.MoveTowards</code> .                                                      |
| Prosedural                 | Dibuat otomatis oleh rumus/kode, bukan digambar atau ditaruh satu per satu (contoh: langit bintang kami).                                              |
| Meta UI                    | Efek layar sesaat: fade hitam, kilat putih, titik bidik. Bukan panel informasi — dan satu-satunya yang boleh Screen Space Overlay menurut aturan soal. |
| State (kondisi permainan)  | Semua yang berubah selama bermain: posisi kereta, stempel, pintu, show. Reset = mengembalikan semua state ke awal.                                     |
| MenuItem / script editor   | Script yang menambah menu di Unity Editor untuk MEMBANGUN scene (generator). Tidak ikut ke build game.                                                 |

|                  |                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebGL dan Brotli | WebGL = build yang jalan di browser. Brotli = kompresi supaya unduhan kecil dan cepat.                                                               |
| MCP Unity        | Alat produksi yang kami pakai: AI mengendalikan Unity Editor (membuat objek, mengatur komponen) lewat protokol MCP. Alat, bukan pengganti pemahaman. |

# Peran dan Tanggung Jawab Tim

Setiap bagian punya penanggung jawab utama, tetapi semuanya dikerjakan sebagai tim: keputusan konsep diambil bersama, dan setiap orang wajib bisa menjelaskan bagiannya sendiri di depan dosen.

## Antonius Harry — 25120300031

### KETUA DAN SISTEM REL

- **Arti peran:** memimpin arah proyek, membuka dan menutup presentasi, memegang sistem paling struktural: jalur rel.
- **Di proyek ini:** Section 1 Hutan Sihir (piknik teddy, api unggun, kunang-kunang, percabangan Jalur Beruang), desain rute rel 80 titik yang di-bake jadi 755 waypoint, lorong bawah tanah (portal gua S4, lorong air, portal putih S5), serta narasi optimasi WebGL.
- **Area kode:** `Assets/Editor/WahanaLayout.cs`, `Assets/Scripts/PusatWahana.cs`, script show S1, `Assets/Editor/ItchWebGLBuildUtility.cs`.
- **Bagian presentasi:** slide 1-3 (buka), 7-8, 13-16 (tutup); pandu QnA.

## Izhar Rahman Dwiputra — 25120300032

### DUNIA, ONBOARDING, KERETA, DAN UI

- **Arti peran:** integrator — merakit dunia tempat semua section hidup dan sistem yang mengantar pemain: kereta dan UI.
- **Di proyek ini:** dunia malam (langit prosedural, bianglala, lampu taman, jangkrik), lobby dan mekanik tiket, Mode Jalan Kaki, kereta kencana beserta pacing-nya, seluruh UI World Space, dan alur build lewat MCP Unity.
- **Area kode:** `KeretaMover.cs`, `InteraksiRaycast.cs`, `ObjekInteraksi.cs`, `ZonaTrigger.cs`, `RideStatusUI.cs`, `Billboard.cs`, `ModeJalanKaki.cs`, generator `SihirMalam.cs`, `OnboardingFinal.cs`.
- **Bagian presentasi:** slide 4-6; pegang keyboard saat demo.

## Deva Agriani — 25120300026

### SECTION 2: DALAM KOTAK MUSIK

- **Arti peran:** pemilik satu dunia tematik — bertanggung jawab konsep visual, isi, dan interaksinya.
- **Di proyek ini:** kotak musik raksasa: gigi emas berputar, penari di piringan, band monster, penonton manusia salju, salju turun, interaksi wind-up.
- **Area kode:** `Assets/Editor/SihirS2.cs` (generator), `PutarPelan.cs`, `GoyangRitmis.cs`, `SaljuJatuh.cs`, `AksiWindUpS2.cs`.
- **Bagian presentasi:** slide 9; narasi S2 saat demo.

## Halimah Sukmawati — 25120300037

### SECTION 3: KAMAR ANAK TERBENGKALAI

- **Arti peran:** pemilik section horor — mengatur ketegangan lewat cahaya, gerakan halus, dan waktu.
- **Di proyek ini:** kamar skala raksasa (boneka porselen retak, balok ABC, kursi goyang), mata di kegelapan, kepala menoleh, sekuens seram empat tahap, siluet hantu, kotak musik rusak.
- **Area kode:** `Assets/Editor/SihirS3.cs` , `SekuensKamarS3.cs` , `KepalaMenatap.cs` , `MataKegelapan.cs` , `SiluetLewatS3.cs` , `AksiKotakMusikS3.cs` .
- **Bagian presentasi:** slide 10; narasi S3 saat demo.

## Dharma Satriadi — 25120300030

### SECTION 4: AKUARIUM MAINAN RAKSASA

- **Arti peran:** pemilik section bawah laut — satu-satunya section di bawah tanah, sekaligus penjaga tema audio.
- **Di proyek ini:** gua akuarium (kawanan ikan, ubur-ubur, gelembung, kapal karam, kastil), siluet anak di balik kaca, interaksi ketuk kaca, dan pemetaan musik per section beserta lisensinya.
- **Area kode:** `Assets/Editor/SihirS4.cs` , `IkanKawanan.cs` , `GelembungNaik.cs` , `ScrollUV.cs` , `AksiKetukKaca.cs` , `SuasanaZona.cs` , `Assets/Audio/Musik/SUMBER_MUSIK.md` .
- **Bagian presentasi:** slide 11; narasi S4 saat demo.

## Dimas Kurniawan — 25120300016

### SECTION 5: LANGIT KAMAR DAN ENDING

- **Arti peran:** pemilik finale — section terakhir plus penutup cerita.
- **Di proyek ini:** langit kamar (mobile planet, stiker bintang glow, roket orbit, band alien), interaksi Encore, efek portal putih, dan ending "anak tertidur" yang menutup benang merah.
- **Area kode:** `Assets/Editor/SihirS5.cs` , `Assets/Editor/Portals5.cs` , `RoketOrbit.cs` , `TwinkleS5.cs` , `AksiEncoreS5.cs` , `EfekPortals5.cs` , `EndingKamarS5.cs` .
- **Bagian presentasi:** slide 12; narasi S5 dan ending saat demo.

# Peta Proyek

Semua yang penting ada di folder `Assets/`. Scene gameplay hanya SATU.

Assets/

- Scenes/
  - UAS\_Main.unity
- Scripts/
  - KeretaMover.cs
  - PusatWahana.cs
  - InteraksiRaycast.cs
  - ObjekInteraksi.cs
  - ZonaTrigger.cs
  - RideStatusUI.cs
  - SekuensKamarS3.cs
  - EfekPortalS5.cs
  - EndingKamarS5.cs
  - ... (animasi: GoyangRitmis, PutarPelan, IkanKawanan, RoketOrbit, TwinkleS5, KunangDomino, SaljuJatuh, dst.)
  - Dosen/
    - SimpleCharacterController.cs
- Editor/
  - WahanaLayout.cs
  - WahanaRebuilder.cs
  - SihirS2..S5.cs, SihirMalam.cs, OnboardingFinal.cs, PortalS5.cs
  - TemenDresser.cs
  - ItchWebGLBuildUtility.cs
- Audio/
  - Musik/ + SUMBER\_MUSIK.md
  - SFX/ + T7\_SFX\_SOURCES.md
- Prefabs/ Models/ Temen/
- Generated/

satu-satunya scene ride (aturan brief: 1 scene)

52 script runtime – logika wahana

jantung wahana: kereta ikut waypoint

hub pusat + reset ride

mata dan tangan pemain (raycast + E)

benda yang bisa di-interaksi (11 mode)

zona reaksi (6 mode)

panel status + stempel di kereta

show horor 4 tahap (coroutine)

kilat putih portal (meta-UI)

ending "anak tertidur"

7 script pola dari recap dosen

first-person controller

generator scene – 66 menu Tools/Wahana

tabel 80 titik rute utama + resampler waypoint

struktur dunia (ground, shell, rel)

penempatan boneka per anggota

pengaturan + build WebGL itch.io

BGM per section + lisensi (WAJIB baca)

aset visual dan boneka tiap anggota

mesh hasil generator (jangan diedit manual)

| File penting                       | Kenapa penting                                                                |
|------------------------------------|-------------------------------------------------------------------------------|
| Assets/Scenes/UAS_Main.unity       | Scene ride yang di-build. Scene lain di folder Models hanya demo aset.        |
| Assets/Audio/Musik/SUMBER_MUSIK.md | Daftar musik + blok kredit CC-BY yang WAJIB tampil di itch.io dan slide 16.   |
| docs/RULES-UAS.md                  | Rangkuman aturan soal, rubrik, bonus, dan penalti.                            |
| dokumentasi/                       | Deck, skrip presentasi, shot list, dan buku ini.                              |
| Packages/manifest.json             | JANGAN diubah/di-commit — berisi path package lokal alat MCP di laptop Izhar. |



# Urutan Belajar yang Disarankan

---

Ikuti urutan ini kalau mulai dari nol — tiap langkah memakai pengetahuan langkah sebelumnya.

- 1 **SimpleCharacterController.cs** (folder Dosen) — *kenapa dulu*: semua berawal dari pemain yang bisa berjalan dan menoleh. Gerak WASD + kamera mouse + lompat.
- 2 **InteraksiRaycast.cs** lalu **ObjekInteraksi.cs** — *kenapa*: ini "mata dan tangan" pemain: sinar dari tengah layar mendeteksi objek, E menjalankan aksinya, objek menyala saat ditatap.
- 3 **ZonaTrigger.cs** — *kenapa*: kebalikan dari raycast: dunia yang bereaksi saat pemain/kereta masuk area (stempel, pintu, pelan, show).
- 4 **KeretaMover.cs** — *kenapa*: jantung wahana. Baca bagian `Update()` : MoveTowards ke waypoint, batas kecepatan, kontrol W/S, berhenti di percabangan.
- 5 **PusatWahana.cs** — *kenapa*: hub yang menghubungkan semuanya + `ResetSemua()` untuk replay tanpa load scene.
- 6 **RideStatusUI.cs** + **Billboard.cs** — *kenapa*: UI World Space adalah aturan wajib soal; pahami stempel, progress, dan cara papan menghadap pemain.
- 7 **Script section milikmu** (S1-S5: show, animasi, portal, ending) — *kenapa*: inilah bagian yang kamu ceritakan di slide dan demo.
- 8 **Generator editor + build** (WahanaLayout, WahanaRebuilder, ItchWebGLBuildUtility) — *kenapa terakhir*: cara dunia dirakit dan dikirim; penting untuk pertanyaan "bagaimana kalian membangun ini".

# Konsep dan Letaknya di Kode

Pola tiap bagian: **apa dan kenapa, di mana** (file:baris), **kode asli, penjelasan**, kotak **"Kalau ditanya"**, dan **penanggung jawab** yang wajib bisa menjelaskannya.

## 5.1 First-Person Controller dan Mode Jalan Kaki

**Apa dan kenapa:** pemain bergerak WASD + kamera mouse memakai CharacterController — syarat first-person soal. Mode Jalan Kaki (tuas staf di lobby) membuka semua pintu section untuk ditelusuri tanpa kereta.

Assets/Scripts/Dosen/SimpleCharacterController.cs : 63-74

```
private void HandleMovement()
{
    float horizontalInput = Input.GetAxisRaw("Horizontal");
    float verticalInput = Input.GetAxisRaw("Vertical");

    Vector3 inputDirection = new Vector3(horizontalInput, 0f, verticalInput);
    inputDirection = Vector3.ClampMagnitude(inputDirection, 1f);

    Vector3 moveDirection = transform.right * inputDirection.x + transform.forward *
inputDirection.z;
    float currentSpeed = Input.GetKey(KeyCode.LeftShift) ? _moveSpeed * _sprintMultiplier
: _moveSpeed;

    characterController.Move(moveDirection * currentSpeed * Time.deltaTime);
}
```

**Penjelasan:** input dibaca sebagai angka -1..1, diubah jadi arah relatif badan pemain ( transform.right/forward ), lalu characterController.Move menggerakkan sambil menabrak dinding dengan benar. ClampMagnitude mencegah gerak diagonal jadi lebih cepat. Mode Jalan Kaki ada di ModeJalanKaki.cs:23 — satu properti statis Aktif yang dibaca ZonaTrigger dan gerbang tiket.

### Kalau ditanya: "Kenapa pakai CharacterController, bukan Rigidbody?"

CharacterController dirancang untuk gerak karakter: menempel lantai, tidak terpental fisika, dan tidak menembus dinding — pas untuk first-person. Rigidbody kami pakai justru di kereta (kinematik) untuk kebutuhan trigger.

Penanggung jawab: Izhar

## 5.2 Kereta Mengikuti Rel: Waypoint + MoveTowards

**Apa dan kenapa:** kereta berjalan dari waypoint ke waypoint (755 titik) memakai `Vector3.MoveTowards` — sederhana, pasti tidak keluar jalur, dan murah untuk WebGL. Badan kereta menoleh halus memakai `Slerp` eksponensial.

Assets/Scripts/KeretaMover.cs : 377-399 (dipadatkan)

```
// Gerak lurus mendekati waypoint tujuan (pola dosen MoveTowards).
transform.position = Vector3.MoveTowards(
    transform.position,
    tujuan.position,
    kecepatan * Time.deltaTime);

Vector3 arah = (tujuan.position - transform.position).normalized;
if (arah != Vector3.zero)
{
    // Slerp eksponensial: arah hadap mengejar arah tujuan secara halus,
    // frame-rate independent — belokan mulus, kereta ikut menunduk di gua.
    float bobotBelok = 1f - Mathf.Exp(-_kecepatanBelok * Time.deltaTime);
    arahHadap = Vector3.Slerp(arahHadap, arah, bobotBelok);
    transform.rotation = Quaternion.LookRotation(arahHadap);
}
```

**Penjelasan:** tiap frame kereta bergeser sejauh `kecepatan x deltaTime` ke titik tujuan; begitu posisinya PERSIS sama (jaminan `MoveTowards`), `SampaiWaypoint()` (baris 415) memilih titik berikutnya. Batas kecepatan bertingkat (baris 344-357): cabang lalu zona lambat lalu maksimum koridor; W/S menambah/mengurangi dengan akselerasi (baris 362-374). Di depan percabangan S1 kereta berhenti MENUNGGU INPUT, bukan timer (baris 420-439).

**Penting untuk QnA — nilai scene vs default script:** angka di slide (normal 2.0, lambat 1.1) adalah nilai LIVE di Inspector scene, di-set otomatis generator (`WahanaRebuilder.cs:30`). Default di file script berbeda (2.5/1.2 di baris 32-33) — nilai Inspector selalu meng-override default script; itu memang cara kerja `[SerializeField]`. Komentar ringkasan di atas `KeretaMover.cs` (baris 3-8) juga masih menceritakan prototipe awal (78 waypoint, cabang S2, stop S3). Kalau dosen membukanya, jawab tenang: "itu komentar desain awal; desain final terbaca di nilai scene — 755 waypoint, cabang di S1, dan stop bertimer S3 kami nonaktifkan saat tuning pacing."

### Kalau ditanya: "Kenapa tidak pakai physics atau spline runtime?"

Ride kereta harus deterministik: penumpang tidak boleh keluar rel gara-gara tabrakan fisika. `MoveTowards` menjamin posisi selalu di garis antar waypoint. Kehalusan tikungan kami selesaikan di tahap desain rel (80 titik jadi 755 waypoint rapat), bukan di runtime — jadi hasilnya halus TANPA biaya komputasi spline di browser.

Penanggung jawab: Izhar (gerak) dan Harry (rute)

## 5.3 Interaksi Raycast: 11 Mode + Highlight Glow

**Apa dan kenapa:** `InteraksiRaycast.cs` menembakkan sinar dari tengah layar (baris 212: `Physics.Raycast`). Kalau kena `ObjekInteraksi`, objek diberi highlight; tekan E menjalankan aksinya sesuai mode 0-10 (naik kereta, mulai, tuas cabang, tiket, pintu, sampai aksi kustom section lewat kontrak `IAksiInteraksi`).

**Assets/Scripts/ObjekInteraksi.cs : 100-115 (highlight saat ditatap)**

```
if (_renderObjek != null)
{
    Material material = _renderObjek.material;
    if (dilihat)
    {
        material.EnableKeyword("_EMISSION"); // nyalakan emission
        material.SetColor("_EmissionColor", _warnaAsli * _intensitasGlow);
    }
    else
    {
        material.SetColor("_EmissionColor", Color.black); // matikan glow
    }
}

// Efek "pop": sedikit membesar saat dilihat
transform.localScale = dilihat ? _skalaAsli * _faktorMembesar : _skalaAsli;
```

**Penjelasan:** glow memakai warna asli objek dikali intensitas 0.3 (baris 37) supaya halus, bukan putih menyilaukan; skala 1.05 memberi kesan "hidup". Mode aksi didefinisikan di komentar kelas (baris 6-14) dan dieksekusi `Interact()` (baris 122): mode ringan ditangani lokal, mode wahana memanggil hub `PusatWahana`, mode 10 memanggil `IAksiInteraksi` di objek yang sama — cara tiap section menambah aksi unik (wind-up, ketuk kaca, encore) tanpa mengubah sistem pusat.

### Kalau ditanya: "Kenapa satu script untuk semua interaksi, bukan script per objek?"

Supaya konsisten dan mudah dirawat: highlight, suara, dan prompt E seragam di semua objek. Yang berbeda cukup angka mode di Inspector, dan untuk aksi yang benar-benar unik ada mode 10 yang memanggil script kecil khusus section (pola interface).

**Penanggung jawab:** Izhar (sistem); aksi mode 10 masing-masing pemilik section

## 5.4 ZonaTrigger: 6 Mode Reaksi Dunia

**Apa dan kenapa:** kotak-kotak tak terlihat ber-Collider isTrigger. Saat pemain atau kereta masuk, dunia bereaksi: stempel (0), pintu buka-tutup (1), kereta pelan (2), mulai show (3), ubah teks status (4), tandai sisi rute (5).

Assets/Scripts/ZonaTrigger.cs : 81-92

```
private bool CocokTag(Collider other)
{
    if (other.CompareTag(_tagPemicu)) return true;
    Rigidbody rb = other.attachedRigidbody;
    if (rb != null && rb.CompareTag(_tagPemicu)) return true;

    // Mode Jalan Kaki (backstage tour): semua zona pintu (mode 1) ikut merespons
    // Player yang jalan kaki, supaya pintu section bisa dilewati tanpa kereta.
    if (_mode == 1 && ModeJalanKaki.Aktif && other.CompareTag("Player")) return true;

    return false;
}
```

**Penjelasan:** trik pentingnya di `attachedRigidbody` : collider kereta ada di lima anak (Bak depan/belakang/kiri/kanan/lantai) yang tidak ber-tag, sedangkan tag "Kereta" + Rigidbody kinematik ada di ROOT — jadi pengecekan naik ke Rigidbody induknya. Pintu memakai counter `_dalamZona` (baris 45): baru menutup saat SEMUA collider kereta keluar plus jeda, supaya tidak berkedip saat kereta panjang melintas. Zona lambat berpasangan: masuk = pelan, keluar = normal (baris 188-205).

### Kalau ditanya: "Kenapa kereta perlu Rigidbody padahal tidak pakai fisika?"

Aturan Unity: event trigger hanya terkirim kalau minimal salah satu pihak punya Rigidbody. Kereta kami beri Rigidbody kinematik — dianggap "ada" oleh sistem fisika tapi posisinya tetap 100 persen dikontrol script.

Penanggung jawab: Izhar (sistem); contoh stempel dan sisi rute: Deva

## 5.5 UI World Space: Panel Kereta, Stempel, Billboard

**Apa dan kenapa:** aturan soal mewajibkan UI utama World Space. Panel di kereta ( `RideStatusUI` ) menampilkan status, 6 stempel, dan progress bar; papan ringkasan muncul di akhir; label section memakai `Billboard` agar selalu menghadap pemain.

`Assets/Scripts/RideStatusUI.cs` : 153-180 (dipadatkan)

```
public void TandaiStempel(int index)
{
    if (index < 0 || index >= stempelKena.Length) { ... return; }
    if (stempelKena[index]) return; // sudah kena: jangan dobel warna/sfx

    stempelKena[index] = true;

    // Nyalakan warna emas pada icon stempel.
    if (_stempel[index] != null)
        _stempel[index].color = _warnaStempelAktif;

    if (_sfxStempel != null)
        _sfxStempel.Play(); // feedback audio saat stempel kena
}
```

**Penjelasan:** stempel index 0-4 = section S1-S5 (dipicu `ZonaTrigger` mode 0), index 5 = bintang emas yang menyala lewat `TandaiSisi` (baris 202-220) bila kedua rute S1 sudah pernah dilalui — progresnya sengaja bertahan antar-ride, jadi normalnya butuh DUA ride (satu lewat tiap rute). Progress bar memakai `Image.fillAmount` (baris 195). `Billboard.cs:39` menyamakan rotasi label dengan rotasi kamera. Screen Space Overlay hanya untuk meta-UI yang memang diizinkan: fade layar, kilat portal, crosshair.

**Awas komentar lama:** komentar di `RideStatusUI.cs:9-11` dan label mode 5 di `ZonaTrigger.cs:11` masih menulis "dua sisi panggung S2" — nama fitur versi awal. Posisi zona nyatanya ( `Z_SisiKiri` / `Z_SisiKanan` ) ada di Section 1: satu di rute utama, satu di jalur beruang. Kalau dosen menunjuk komentar itu, jelaskan riwayatnya dengan tenang.

### Kalau ditanya: "Tunjukkan mana UI utama dan mana meta-UI."

UI utama (World Space): panel status + stempel + progress di kereta, papan ringkasan akhir, papan info, label section. Meta-UI (Overlay, diizinkan aturan): fade hitam saat transisi, kilat putih portal S5, crosshair. Objective dan status TIDAK pernah di Overlay.

Penanggung jawab: Izhar

## 5.6 PusatWahana: Hub dan Replay Tanpa Load Scene

**Apa dan kenapa:** satu "resepsionis" di root Wahana: script lain cukup menemukan hub ini untuk mencapai kereta, UI, show, papan — tidak perlu saling drag reference. Juga pusat reset ride.

Assets/Scripts/PusatWahana.cs : 102-126 (dipadatkan)

```
public void ResetSemua()
{
    // Guard null satu-satu supaya reset tetap jalan walau ada sistem hilang
    if (_kereta != null) _kereta.ResetKeAwal();
    if (_statusUI != null) _statusUI.ResetStempel();
    if (_sequence != null) _sequence.ResetSequence();
    if (_ringkasan != null) _ringkasan.ResetRingkasan();

    for (int i = 0; i < _semuaPintu.Length; i++)
        if (_semuaPintu[i] != null) _semuaPintu[i].ResetPintu();

    // Zona "hanya sekali" dibuka lagi supaya ride kedua tetap berfungsi
    for (int i = 0; i < _semuaZona.Length; i++)
        if (_semuaZona[i] != null) _semuaZona[i].ResetZona();
}
```

**Penjelasan:** brief menuntut SATU scene, jadi replay tidak boleh `LoadScene`. Solusinya: setiap sistem punya method reset sendiri, dan hub memanggil semuanya — kereta pulang ke boarding, stempel kosong, pintu tertutup, zona siap dipicu lagi. Tiket sengaja TIDAK di-reset di sini: tiket hangus saat kereta berangkat (`PakaiTiket`, baris 91), jadi tiket yang belum terpakai tetap berlaku. Fallback auto-find di Awake (baris 45-64) adalah pola seluruh project karena alat MCP tidak bisa mengisi reference Inspector.

### Kalau ditanya: "Bagaimana kalian memastikan ride kedua tidak rusak?"

Semua state yang berubah selama ride punya jalur pulang: `ResetSemua` mengembalikan kereta, stempel, show, papan, pintu, dan zona sekali-pakai. Kami mengetesnya dengan menjalankan ride dua kali berturut-turut tanpa keluar Play mode.

Penanggung jawab: Harry

## 5.7 Desain Rel: 80 Titik Menjadi 755 Waypoint

**Apa dan kenapa:** rel dirancang sebagai tabel 80 titik bernama (boarding, pintu section, portal gua) untuk rute utama — plus 6 titik untuk cabang beruang — di script editor, lalu di-"bake" menjadi 755 waypoint rapat berjarak 0,5 unit dengan tikungan busur halus. Sekali jalan, hasilnya selalu sama.

Assets/Editor/WahanaLayout.cs : 460-467

```
/// Resample node -> List titik world dengan busur fillet di tiap belokan.  
/// closed = true: loop tertutup (elemen terakhir menyambung node terakhir -> pertama).  
/// closed = false: polyline terbuka (cabang WK).  
/// Titik PERTAMA selalu eksak = node[0].pos (WP_0). Spacing dinormalisasi  
/// total/round(total/SpacingTarget). Y node yProfil di-override profil ketinggian.  
public static List<Vector3> Resample(Node[] nodes, bool closed)
```

**Penjelasan:** tiap belokan dipotong busur berjari-jari (fillet — istilah teknisnya; cukup sebut "busur halus") supaya kereta tidak menikung patah; profil ketinggian menurunkan rel mulus ke gua S4 (kedalaman -6) lalu menaikkannya lagi. Hasil bake berupa GameObject WP\_0..WP\_754 di bawah JalurUtama, plus 42 titik cabang Jalur Beruang. Jarak 0,5 unit antar titik dipilih supaya rotasi kereta selalu punya target dekat — belokan terasa seperti rel sungguhan. SpacingTarget ada di baris 23. Di file juga masih ada tabel 5 titik cabang S2 versi awal ( BuildNodeKiri, baris 212) yang kami nonaktifkan saat rebalancing rute — cabang final di S1.

### Kalau ditanya: "Dari mana angka 755? Kalian menaruh 755 titik manual?"

Tidak — kami hanya mendesain 80 titik penting untuk rute utama (belokan, pintu, portal; cabang beruang cuma 6 titik lagi). Program editor menghitung panjang total lintasan, membaginya per 0,5 unit, dan menempatkan 755 waypoint otomatis, termasuk busur halus di tiap tikungan. Kalau layout berubah, tinggal jalankan ulang menunya.

Penanggung jawab: Harry



## 5.8 Show Sequence dengan Coroutine (S3, Empat Tahap)

**Apa dan kenapa:** show yang berjalan bertahap dalam waktu (redup, blackout, kejut, pulih) tidak bisa ditulis di Update biasa tanpa jadi ruwet. Coroutine membuat urutannya terbaca seperti naskah.

Assets/Scripts/SekuensKamarS3.cs : 98-130 (dipadatkan)

```
// --- Tahap 1: meredup ~2 dtk ---
yield return.LerpIntensitas(lampu, intensitasAsal, _faktorRedup, _durasiRedup);

// --- Tahap 2: BLACKOUT 1.5 dtk ---
for (int i = 0; i < lampu.Count; i++)
    if (lampu[i] != null) lampu[i].enabled = false;
yield return new WaitForSeconds(_durasiBlackout);

// --- Tahap 3: nyala remang + SWAP boneka + kejut hero + audio ---
if (_bonekaSetA != null) _bonekaSetA.SetActive(false);
if (_bonekaSetB != null) _bonekaSetB.SetActive(true); // lebih dekat rel!
if (_heroKepala != null) _heroKepala.PaksaMenatap(true);
if (_bisikan != null) _bisikan.Play();
if (_musikHoror != null) _musikHoror.Play();
```

**Penjelasan:** `yield return` menjeda fungsi tanpa membekukan game. Tahap 4 (baris 137-142) mengembalikan intensitas lampu ke nilai asal yang di-snapshot saat trigger. Trik horrornya klasik dark ride: saat gelap total, set boneka A diganti set B yang posisinya lebih dekat rel — penumpang merasa bonekanya "berpindah". Show dipicu `PemicuKereta` (zona khusus kereta) lewat kontrak `IAksiInteraksi.Jalankan()` (baris 81), dengan guard `_sedangJalan` supaya tidak doble.

**Detail yang sering ditanya:** sekuens hanya terpicu KERETA — `PemicuKereta` menyaring tag "Kereta", jadi saat Mode Jalan Kaki show S3 TIDAK menyala (sengaja: mode itu untuk inspeksi). Pencarian mata kegelapan dengan `FindObjectsByType` (baris 107) berjalan sekali per pemicu di dalam coroutine, bukan tiap frame — aman untuk WebGL.

### Kalau ditanya: "Apa bedanya coroutine dengan Update? Kenapa tidak pakai animasi biasa?"

Update dipanggil tiap frame dan cocok untuk gerak kontinu; coroutine bisa "menunggu 1,5 detik lalu lanjut" — pas untuk naskah bertahap. Animasi clip hanya menggerakkan satu objek; sekuens ini mengatur BANYAK hal sekaligus (semua lampu S3, dua set boneka, kepala hero, dua audio) dalam satu urutan waktu.

Penanggung jawab: Halimah (S3); pola serupa dipakai show S1 (Harry)

## 5.9 Variasi Animasi: Animator vs Animasi Script

**Apa dan kenapa:** syarat soal: minimal 3 animasi boneka yang berbeda. Kami memakai dua keluarga: Animator (clip: boneka menari, pintu) dan animasi script yang menggerakkan Transform langsung — lebih dari sepuluh macam gerak berbeda di lima section.

Assets/Scripts/GoyangRitmish.cs : 28-32

```
private void Update()
{
    float sudut = Mathf.Sin(Time.time * _tempo * Multiplier + _fase) * _amplitudo;
    transform.localRotation = _rotasiAwal * Quaternion.AngleAxis(sudut, _sumbu);
}
```

**Penjelasan:** satu baris sinus = goyangan ritmis manusia salju S2 dan band alien S5; `_fase` diambil dari posisi dunia (baris 25) supaya tiap boneka berayun beda tanpa Random. Keluarga lainnya: `PutarPelan` (gigi dan mobile berputar), `KepalaMenatap.cs:73` ( `Quaternion.RotateTowards` 20 derajat/detik — kepala menoleh pelan), `IkanKawanan` (orbit elips), `RoketOrbit`, `TwinkleS5` (kelip), `KunangDomino` (menyala berurutan), `SaljuJatuh`, `GelembungNaik`. Boneka ber-rig memakai `BonekaAnimator` / `UAS_TeddyAnimator` dengan Animator Controller.

### Kalau ditanya: "Kapan kalian memilih Animator, kapan script?"

Animator untuk gerakan karakter yang punya clip (menari, jalan) — bisa diatur transisinya. Script untuk gerak berulang yang dihasilkan rumus (putar, goyang, orbit, kelip): lebih ringan, tidak butuh clip, dan mudah diberi variasi fase per objek. Keduanya tampil di gameplay, jadi syarat "3 animasi berbeda" terlampaui jauh.

**Dua kejujuran kecil (khusus Deva):** (1) S2 sepenuhnya animasi script — Animator ada di boneka teddy S1 ( `BonekaAnimator` / `UAS_TeddyAnimator` ) dan pintu; kalau ditanya "mana Animator di section-mu", arahkan ke sana dengan jujur. (2) Goyangan manusia salju itu ritmis bertempo tetap — TIDAK menganalisis beat musik; kalau ditanya "sinkron ke musik?", jawab: temponya dipilih supaya terasa senada dengan waltz-nya, bukan sinkronisasi beat.

Penanggung jawab: Deva (utama); contoh per section: Halimah, Dharma, Dimas, Harry

## 5.10 Audio per Section dan Lisensi Musik

**Apa dan kenapa:** tiap section punya BGM dan ambience sendiri (hutan, waltz kotak musik, drone horor, bawah laut, angkasa) plus SFX interaksi — feedback audio adalah syarat soal. Semua musik legal: CC0, kecuali satu track CC-BY yang wajib kredit.

**Assets/Scripts/SuasanaZona.cs : 20-32 (field profil suasana)**

```
[Header("Mode (0 = set profil target, 1 = restore default)")]
[SerializeField, Range(0, 1)] private int _mode = 0;

[Header("Deteksi (WAJIB \"Kereta\" untuk on-ride)")]
[SerializeField] private string _tagPemicu = "Kereta";

[Header("Durasi transisi (detik)")]
[SerializeField] private float _durasi = 2f;

[Header("Profil target — Fog (dipakai saat mode 0)")]
[SerializeField] private Color _fogColor = new Color(0.01f, 0.02f, 0.05f, 1f);
```

**Penjelasan:** `SuasanaZona` melengkapi audio: saat kereta masuk zona, fog + warna ambient + volume ambience di-lerp halus ke profil section (masuk gua jadi biru gelap, keluar kembali malam normal). BGM per section berupa AudioSource loop yang ditata generator. Lisensi terdokumentasi di `Assets/Audio/Musik/SUMBER_MUSIK.md`.

**Kredit wajib (CC-BY 4.0) — harus tampil di halaman itch.io dan slide 16:**

"Fireflies and Stardust" Kevin MacLeod (incompetech.com)  
Licensed under Creative Commons: By Attribution 4.0 License  
<http://creativecommons.org/licenses/by/4.0/>

**Peta track (hafalan Dharma):** track CC-BY di atas adalah BGM **Section 1 Hutan** — bukan S4. Lobby: "Barroom Ballet" · S2: "A Waltz For Naseem" · S3: "The Abyss" · S4: "Underwater Theme II" (CC0, Cleyton Kauffman) · S5: "Space Ambience" — selain yang CC-BY, semuanya CC0 (FreePD/OpenGameArt). Catatan SuasanaZona: hanya bereaksi ke kereta (tag "Kereta"), jadi saat Mode Jalan Kaki fog/suasana tidak berubah — disengaja untuk mode inspeksi.

**Kalau ditanya: "Kenapa musik yang satu ini wajib kredit, yang lain tidak?"**

Lisensi berbeda: track lain CC0 (public domain, bebas tanpa atribusi), sedangkan "Fireflies and Stardust" berlisensi Creative Commons By-Attribution — boleh dipakai gratis ASAL mencantumkan pencipta. Kami cantumkan persis format resminya.

**Penanggung jawab: Dharma**

## 5.11 Generator Editor dan Kejujuran Soal AI

**Apa dan kenapa:** dunia tidak dirakit tangan satu-satu: 66 menu `Tools/Wahana` membangunnya dari kode editor — deterministik (seed tetap) dan idempoten (jalankan ulang = hasil identik). Ini juga jawaban jujur untuk pertanyaan "pakai AI?".

`Assets/Editor/WahanaRebuilder.cs` : 8-20 (aturan generator)

```
/// GENERATOR STRUKTURAL WAHANA (Stage 1-4) — 6 MenuItem di Tools/Wahana.  
/// Semua struktural (jalur, ground, shell, tunnel, rel, dressing) dibangun  
/// deterministik dari WahanaLayout. Idempotent: tiap menu hapus output miliknya  
/// dulu (parent GEN_*), lalu bangun ulang dari koordinat ABSOLUT tabel.  
/// - Material baru: shader "Universal Render Pipeline/Lit" (BUKAN Standard = magenta  
URP).  
/// - Seeded System.Random(42).
```

**Penjelasan:** tiap menu menghapus grup `GEN_*` miliknya lalu membangun ulang dari tabel koordinat — revisi aman, tidak ada "sisa sampah". Penempatan acak (rumput, bintang) memakai `System.Random(42)` : acak tapi hasilnya selalu sama. Boneka tiap anggota ditempatkan `TemenDresser.cs` sesuai pembagian tim.

### Kalau ditanya: "Ini dibuat pakai AI, kan? Apa yang kalian kerjakan sendiri?"

Ya — AI kami pakai sebagai alat produksi lewat MCP yang mengendalikan Unity Editor; dosen membebaskan metode, dan yang dinilai kreativitas experience. Yang lahir dari tim: konsep "kita = mainan", desain kelima section dan rutenya, pemilihan aset dan musik, playtest berulang beserta keputusan revisinya (pacing kereta, durasi portal, penempatan boneka). Tiap anggota memahami sistem bagiannya — kalau Bapak ingin menguji, kami siap menjelaskan bagian mana pun.

Penanggung jawab: Harry (ketua) — SEMUA anggota hafal framing ini

## 5.12 Optimasi WebGL

**Apa dan kenapa:** WebGL lag kena penalti sampai -15. Kami menjaga performa dari empat sisi: anggaran renderer, penggabungan mesh, pemangkasan aset, dan kompresi build.

Assets/Editor/ItchWebGLBuildUtility.cs : 139-148

```
private static void ApplyWebGLPlayerSettings()
{
    PlayerSettings.runInBackground = true;

    Type webGLSettingsType = typeof(PlayerSettings).GetNestedType("WebGL",
        BindingFlags.Public | BindingFlags.Static);
    SetStaticEnumProperty(webGLSettingsType, "compressionFormat", "Brotli", fallbackValue:
2);
    SetStaticProperty(webGLSettingsType, "decompressionFallback", true);
    SetStaticProperty(webGLSettingsType, "threadsSupport", false);
    SetStaticProperty(webGLSettingsType, "dataCaching", false);
}
```

**Penjelasan:** (1) Anggaran 450 MeshRenderer aktif ( `WahanaRebuilder.cs:26` ) dipantau menu audit; (2) mesh statis digabung per material lewat menu bake supaya draw call turun; (3) aset dipangkas — paket rumput TriForge dari 643 MB tinggal 4,6 MB (hanya 4 file terpakai), musik dipotong/re-encode ke sekitar 1-3 MB per track; (4) build Brotli = unduhan kecil, dengan decompression fallback supaya tetap jalan di server itch.io. Efek mahal dihindari: kunang dan bintang adalah titik emissive tanpa komponen Light.

### Kalau ditanya: "Bagaimana kalian tahu build-nya tidak berat?"

Kami punya menu audit yang menghitung MeshRenderer aktif terhadap anggaran 450, menguji Play mode dengan Stats, dan mengetes build WebGL langsung di browser. Kalau ada section melewati anggaran, kami gabungkan mesh atau kurangi dekornya sebelum lanjut.

Penanggung jawab: Harry (narasi) dan Izhar (eksekusi build)

## 5.13 Portal Putih dan Ending "Anak Tertidur"

**Apa dan kenapa:** dua momen sinematik S5. Portal: layar memutih + kamera bergetar saat kereta "menembus dinding" ke angkasa — transisi antar dunia tanpa ganti scene. Ending: seluruh section melambat dan meredup saat sang anak tertidur — penutup benang merah.

Assets/Scripts/EfekPortalS5.cs : 67-77

```
/// Dipanggil ZonaPortalS5 saat satu collider kereta masuk zona portal.
public void LaporMasuk()
{
    _dalamZona++;
    if (_fase == Fase.Idle || _fase == Fase.Turun)
    {
        // alpha LANJUT dari nilai berjalan (bukan reset) — robust kalau kereta
        // masuk zona berikutnya saat fade-out belum selesai.
        _fase = Fase.Naik;
    }
}
```

**Penjelasan:** portal memakai state kecil (Idle, Naik, Tahan, Turun) + counter collider kereta; guncangan kamera memakai Perlin noise dan HANYA menyentuh localPosition supaya tidak bentrok dengan script kamera. Overlay putihnya termasuk meta-UI yang diizinkan rubrik (kategori "flash"). Ending ( `EndingKamarS5.cs` ) dipicu beberapa waypoint sebelum keluar S5: siluet kepala anak fade-in lalu "menguap", semua lampu S5 turun ke 35 persen, mobile melambat ke 30 persen, musik ke 40 persen — nilai asal di-snapshot di Awake supaya restore selalu benar, dan sebelum mulai, ending menghentikan dan memulihkan encore dulu ( `AksiEncoreS5.HentikanDanPulihkan` ) supaya lampu tidak "keracunan" nilai strobo.

**Edge case yang disengaja (bekal QnA Dimas):** kalau kereta ditahan (S) tepat di dalam zona portal, layar tetap putih sampai kereta jalan lagi — by design, dan ada failsafe: saat ride selesai atau di-reset, efek dipaksa memudar. Satu lagi: kilat putih memang menutup pandangan SESAAT, tapi itu berbeda dengan "menaruh UI utama di Overlay" — panel status tetap objek World Space di kereta sepanjang waktu.

### Kalau ditanya: "Layar putih itu Screen Space Overlay — bukannya dilarang?"

Yang dilarang adalah UI UTAMA di Overlay. Rubrik eksplisit mengizinkan meta-UI Overlay untuk efek layar seperti fade, flash, dan vignette. Kilat portal masuk kategori itu; semua informasi (status, stempel, objective) tetap di World Space.

Penanggung jawab: Dimas

# Peta Slide ke Kode

Jembatan antara deck dan buku ini: dari slide mana pun, tahu konsep dan file yang membuktikannya.

| Slide | Judul                        | Konsep (Bab 5) | File bukti utama                                          | Pemateri |
|-------|------------------------------|----------------|-----------------------------------------------------------|----------|
| 1     | Judul dan tim                | —              | README.md (roster)                                        | Harry    |
| 2     | Tema dan tujuan              | —              | docs/RULES-UAS.md (Soal 2)                                | Harry    |
| 3     | Alur dan objective           | 5.5, 5.6       | RideStatusUI.cs, PusatWahana.cs                           | Harry    |
| 4     | Dunia malam dan onboarding   | 5.1            | SihirMalam.cs, OnboardingFinal.cs, ModeJalanKaki.cs       | Izhar    |
| 5     | Kereta dan UI World Space    | 5.2, 5.5       | KeretaMover.cs, RideStatusUI.cs                           | Izhar    |
| 6     | Interaksi, trigger, collider | 5.3, 5.4       | InteraksiRaycast.cs, ObjekInteraksi.cs, ZonaTrigger.cs    | Izhar    |
| 7     | Sistem rel dan lorong        | 5.7            | WahanaLayout.cs                                           | Harry    |
| 8     | S1 Hutan Sihir               | 5.8, 5.9       | KunangDomino.cs, LampuFlicker.cs, KeretaMover.cs (cabang) | Harry    |
| 9     | S2 Kotak Musik               | 5.9            | SihirS2.cs, GoyangRitmis.cs, AksiWindUpS2.cs              | Deva     |
| 10    | S3 Kamar Anak                | 5.8            | SekuensKamarS3.cs, KepalaMenatap.cs                       | Halimah  |
| 11    | S4 Akuarium                  | 5.9, 5.10      | IkanKawanan.cs, SuasanaZona.cs, AksiKetukKaca.cs          | Dharma   |
| 12    | S5 dan ending                | 5.13           | EfekPortalS5.cs, EndingKamarS5.cs, AksiEncoreS5.cs        | Dimas    |
| 13    | Reusable dan optimasi        | 5.11, 5.12     | WahanaRebuilder.cs, TemenDresser.cs                       | Harry    |
| 14    | Kendala dan solusi           | 5.6, 5.12      | PusatWahana.cs, ItchWebGLBuildUtility.cs                  | Harry    |
| 15    | Pembagian tugas              | Bab 2          | TemenDresser.cs (pembagian di kode)                       | Harry    |
| 16    | Link dan kredit              | 5.10           | SUMBER_MUSIK.md                                           | Harry    |

# Antisipasi Pertanyaan Dosen

Dua belas pertanyaan yang paling mungkin, dengan jawaban yang bisa dipertahankan. Jawab dengan kata sendiri; angka dan lokasi file boleh dibaca dari buku.

## 1. "Kenapa cuma satu scene? Bagaimana kalau mau main lagi?"

Brief menuntut satu scene, jadi replay tidak memakai `LoadScene`. Tuas reset memanggil `PusatWahana.ResetSemua()` (`PusatWahana.cs:102`) yang memulangkan kereta, mengosongkan stempel, menghentikan show, menutup pintu, dan mempersenjatai ulang semua zona.

## 2. "Kenapa kereta pakai `MoveTowards`, bukan physics atau spline?"

Hasilnya selalu pasti (deterministik) dan murah: posisi selalu di garis waypoint, tidak mungkin terlempar fisika, dan tanpa perhitungan kurva di runtime. Kehalusannya dari desain data — 755 waypoint rapat berjarak 0,5 unit dengan tikungan busur (`WahanaLayout.cs:467`).

## 3. "Tunjukkan UI utama kalian `World Space`. Yang Overlay apa saja?"

UI utama menempel di dunia: panel status + 6 stempel + progress di kereta (`RideStatusUI`), papan ringkasan akhir, papan info, label section. Overlay hanya meta-UI yang diizinkan: fade hitam, kilat putih portal S5, crosshair.

## 4. "Bagaimana kereta tahu harus melambat di depan display?"

Zona lambat (`ZonaTrigger` mode 2): masuk zona memanggil `SetKecepatanLambat()`, keluar memanggil `SetKecepatanNormal()` (`ZonaTrigger.cs:134-138, 201-204`). Kecepatan turun halus dari 2.0 ke 1.1 karena ada ramp akselerasi, bukan langsung patah.

## 5. "Dari mana angka 755 waypoint? Kalian menaruhnya satu-satu?"

Tidak. Kami mendesain 80 titik penting untuk rute utama (plus 6 titik cabang beruang); program editor membagi ulang jalur per 0,5 unit menjadi 755 waypoint plus busur halus di tiap belokan. Ubah desain, jalankan ulang menu, waypoint dibangun ulang identik.

## 6. "Bagaimana percabangan bekerja? Kalau pemain tidak memilih?"

Kereta berhenti penuh beberapa waypoint sebelum titik cabang fisik (berhenti di WP 73, cabang di WP 77; `KeretaMover.cs:420-439`) dan menunggu input: E pada tuas memilih Jalur Beruang, W berangkat lurus. Tidak ada timer paksa — pemain pegang kendali. Kedua rute bergabung lagi di WP 135. Melewati dua-duanya (boleh beda ride) menyalakan bintang emas (`RideStatusUI.TandaiSisi`, baris 202).



### 7. "WebGL kalian berat tidak? Apa buktinya?"

Kami menjaga anggaran 450 MeshRenderer (ada menu audit), menggabung mesh statis per material, memangkas paket aset 643 MB jadi 4,6 MB, memotong audio, dan build Brotli. Kunang dan bintang hanya titik emissive tanpa Light. Diuji langsung di browser sebelum dikumpulkan.

### 8. "Ini dikerjakan pakai AI?"

Ya — sebagai alat produksi: AI mengendalikan Unity Editor lewat MCP, dan dosen membebaskan metode karena yang dinilai kreativitas experience. Konsep cerita, desain section, kurasi aset dan musik, playtest, dan keputusan revisi datang dari tim, dan tiap anggota bisa menjelaskan sistem bagiannya. Kalau Bapak ingin menguji, kami siap menjelaskan bagian mana pun.

### 9. "Musiknya dari mana? Legal?"

Semua berlisensi bebas: mayoritas CC0 dari FreePD dan OpenGameArt, satu track CC-BY 4.0 ("Fireflies and Stardust", Kevin MacLeod) yang kreditnya kami cantumkan di slide penutup dan halaman itch.io — formatnya persis di SUMBER\_MUSIK.md.

### 10. "Kalau pemain turun di tengah ride atau tidak menarik tuas, rusak tidak?"

Tidak. Sebelum berangkat, Q untuk turun dan kereta menunggu; S menahan kecepatan sampai nol dan W melanjutkan; ride selesai otomatis saat kembali ke WP\_0, lalu pemain diturunkan di titik turun. Semua kondisi ada jalur pulangnya.

### 11. "Kenapa kereta punya Rigidbody padahal gerakanya dari script?"

Event trigger Unity butuh Rigidbody minimal di satu pihak. Rigidbody kereta di-set kinematik: terdaftar di sistem fisika (trigger terbaca), tapi posisi tetap dikontrol script. Collider kereta ada di lima anak; zona mengenalinya lewat `attachedRigidbody` (ZonaTrigger.cs:84).

### 12. "Sebutkan tiga animasi berbeda yang terlihat di gameplay."

Lebih dari tiga: penari dan gigi berputar (PutarPelan) plus manusia salju bergoyang (GoyangRitmis) di S2; kepala boneka menoleh (KepalaMenatap) dan sekuens empat tahap di S3; kawanan ikan mengorbit (IkanKawanan) di S4; roket orbit dan bintang berkelip di S5; kunang menyala berurutan di S1; ditambah boneka ber-Animator (teddy piknik).

### 13. "Di kode ada PilihJalurKiri dan mode 3 'jalur kiri panggung S2' — kok tidak terpakai?"

Itu percabangan versi awal di S2. Saat rebalancing rute, percabangan kami pindahkan ke S1 (jalur beruang, mode 9) dan cabang S2 dinonaktifkan cukup lewat data — jumlah waypoint kirinya nol di Inspector — tanpa mengubah kode. Kodenya generik dan tidak mengganggu, jadi dibiarkan; justru bukti fitur bisa dipasang-lepas lewat data. Hal yang sama berlaku untuk stop bertimer S3: durasinya kami nolkan saat tuning pacing.

# Checklist Kesiapan per Anggota

---

## Semua anggota

- ☐ Bisa menceritakan alur ride dari gerbang sampai "Ride Complete" tanpa melihat catatan.
- ☐ Tahu bagian presentasi sendiri dan satu kalimat serah terima ke pemateri berikutnya.
- ☐ Hafal framing jawaban "ini dibuat pakai AI?" (Bab 5.11 dan Bab 7 nomor 8) dengan kata sendiri.
- ☐ Bisa menunjuk minimal SATU file kode bagiannya dan menjelaskan satu method di dalamnya.
- ☐ Sudah memainkan build final minimal satu kali sampai selesai.

## Harry (ketua — buka/tutup, rel, S1)

- ☐ Bab 5.6 (PusatWahana), 5.7 (rel 80 ke 755), 5.11 (generator + framing AI), 5.12 (WebGL).
- ☐ Latihan: "dari mana 755 waypoint?", "bagaimana replay tanpa load scene?", "kenapa build kalian ringan?"
- ☐ Sehari sebelum maju: jalankan menu audit renderer (Tools/Wahana, menu Stats) dan CATAT angka aktualnya — jangan hanya hafal budget 450.
- ☐ Demo: narasi S1 (kunang, cabang Jalur Beruang) dan lorong gua; pandu QnA di akhir (moderasi jawaban maksimal 30 detik).

## Izhar (dunia, onboarding, kereta, UI)

- ☐ Bab 5.1 (controller), 5.2 (KeretaMover), 5.3 (raycast), 5.4 (trigger), 5.5 (UI World Space).
- ☐ Latihan: "kenapa kereta tak bisa keluar jalur?", "mana UI utama vs meta-UI?", "kenapa kereta ber-Rigidbody?"
- ☐ Demo: pegang keyboard; pastikan ambil tiket, tuas start, dan tuas cabang terlihat kamera.

## Deva (S2 Kotak Musik)

- ☐ Bab 5.9 (GoyangRitmis dan kawan-kawan) + cara wind-up memicu show (mode 10, IAksiInteraksi).
- ☐ Latihan: "kenapa tiap manusia salju goyongannya beda?" (fase dari posisi, tanpa Random).
- ☐ Demo: narasi S2 — sebut gigi berputar, penari, dan wind-up dalam satu napas.

## Halimah (S3 Kamar Anak)

- ☐ Bab 5.8 (coroutine empat tahap) + KepalaMenatap (menoleh pelan, clamp yaw).
- ☐ Latihan: "apa itu coroutine?", "trik boneka berpindah saat blackout", dan ingat: sekuens hanya menyala untuk KERETA — saat Mode Jalan Kaki show tidak terpicu (disengaja).
- ☐ Demo: narasi S3 tepat saat sekuens menyala (kereta melambat di zona).

## Dharma (S4 Aquarium + audio)

- ☐ Bab 5.10 (audio dan lisensi — hafal alasan kredit CC-BY DAN peta track: yang CC-BY itu BGM S1 Hutan, bukan S4) + IkanKawanan dan SuasanaZona.
- ☐ Latihan: "kenapa musik wajib kredit?", "bagaimana suasana berubah saat masuk gua?"
- ☐ Demo: narasi S4 — tunjuk siluet anak di balik kaca (momen twist).

## Dimas (S5 + ending)

- ☐ Bab 5.13 (portal putih = meta-UI yang diizinkan; ending anak tertidur) + Encore.
- ☐ Latihan: "kenapa layar putih boleh Overlay?", "apa saja yang meredup saat ending dan bagaimana pulihnya?"
- ☐ Demo: narasi portal masuk S5 sampai "Ride Complete".

**Pegangan terakhir:** kuncinya bukan menghafal jawaban, tapi tenang menunjuk baris kode dan menjelaskan KENAPA-nya. Kalau lupa angka, buka buku ini — dosen menilai pemahaman, bukan hafalan.